1.2.MySQL中的索引

InnoDB存储引擎支持以下几种常见的索引：B+树索引、全文索引、哈希索引，其中比较关键的是B+树索引

1.2.1.B+树索引

InnoDB中的索引自然也是按照B+树来组织的，前面我们说过B+树的叶子节点用来放数据的，但是放什么数据呢？索引自然是要放的，因为B+树的作用本来就是就是为了快速检索数据而提出的一种数据结构，不放索引放什么呢？但是数据库中的表，数据才是我们真正需要的数据，索引只是辅助数据，甚至于一个表可以没有自定义索引。InnoDB中的数据到底是如何组织的？

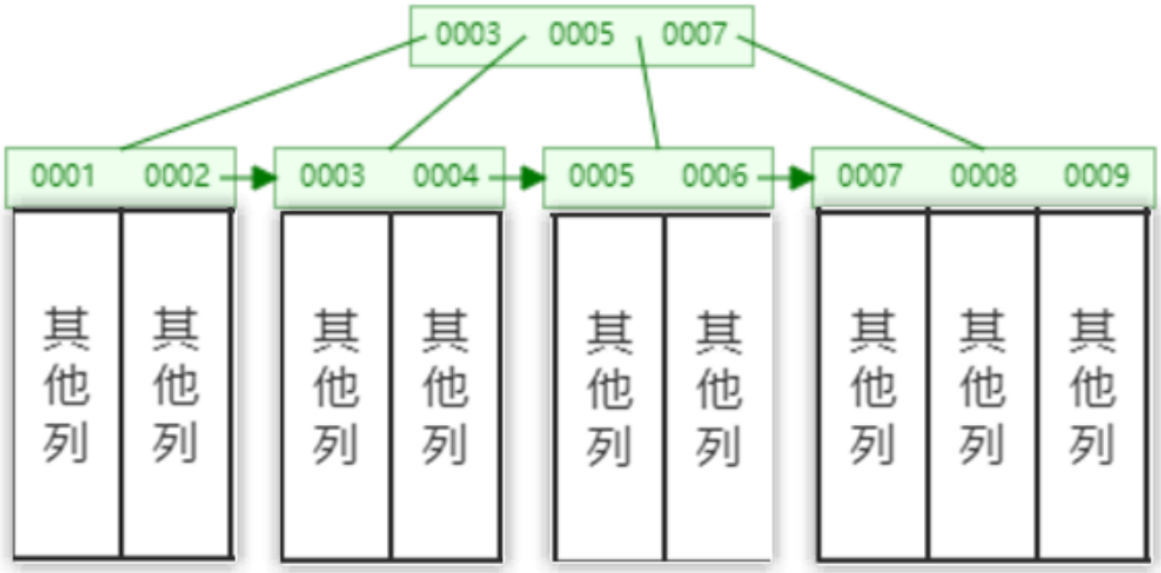
1.2.1.1.聚集索引/聚簇索引

InnoDB中使用了聚集索引，就是将表的主键用来构造一棵B+树，并且将整张表的行记录数据存放在该B+树的叶子节点中。也就是所谓的索引即数据，数据即索引。由于聚集索引是利用表的主键构建的，所以每张表只能拥有一个聚集索引。

聚集索引的叶子节点就是数据页。换句话说，数据页上存放的是完整的每行记录。因此聚集索引的一个优点就是：通过过聚集索引能获取完整的整行数据。另一个优点是：对于主键的排序查找和范围查找速度非常快。

名	类型	长度	小数点	不是 null	
order_id	bigint	50	0	☑	🔑1
user_id	bigint	50	0	☐	
order_status	int	1	0	☐	
pay_status	int	1	0	☐	
shipping_status	int	1	0	☐	

order_id	user_id	order_status	pay_status	shipping_status	goods_id
1	1	0	1	0	13
2	1	0	1	0	13
3	1	0	1	0	13
4	1	0	1	0	13
5	1	0	1	0	13



如果我们没有定义主键呢？MySQL会使用唯一性索引，没有唯一性索引，MySQL也会创建一个隐含列RowID来做主键，然后用这个主键来建立聚集索引。

1.2.1.2.辅助索引/二级索引

聚簇索引只能在搜索条件是主键值时才能发挥作用，因为B+树中的数据都是按照主键进行排序的。

如果我们想以别的列作为搜索条件怎么办？我们一般会建立多个索引，这些索引被称为辅助索引/二级索引。

（每建立一个索引，就有一颗B+树

对于辅助索引(Secondary Index，也称二级索引、非聚集索引)，叶子节点并不包含行记录的全部数据。叶子节点除了包含键值以外，每个叶子节点中的索引行中还包含了一个书签(bookmark)。该书签用来告诉InnoDB存储引擎哪里可以找到与索引相对应的行数据。因此InnoDB存储引擎的辅助索引的书签就是相应行数据的聚集索引键。

名	类型	长度	小数点	不是 null	
id	bigint	22	0	☑	🔑1
order_no	varchar	50	0	☑	
order_note	varchar	100	0	☑	
insert_time	datetime	0	0	☑	
expire_duration	bigint	22	0	☑	
expire_time	datetime	0	0	☑	
order_status	smallint	6	0	☑	

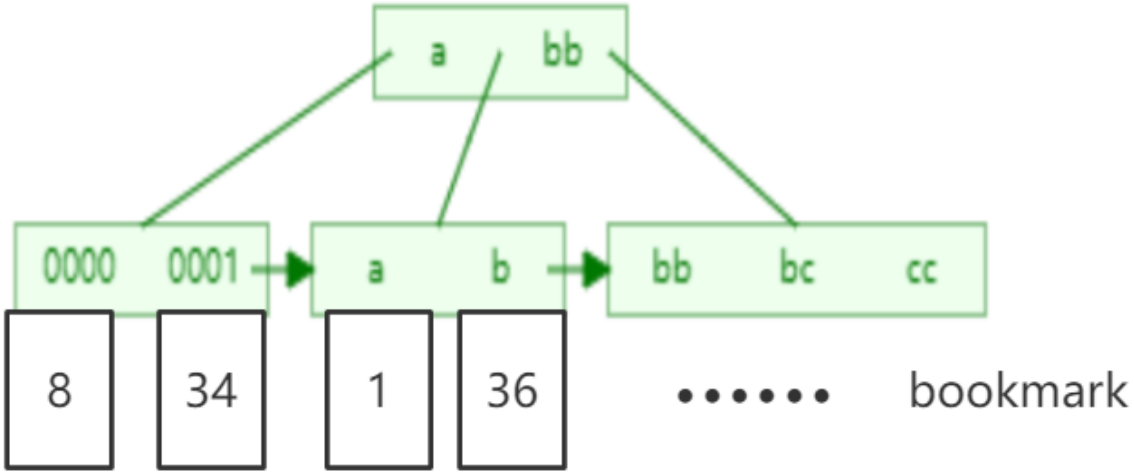
新建 保存 另存为 添加索引 删除索引

栏位 索引 外键 触发器 选项 注释 SQL 预览

名	栏位	索引类型	索引方法
idx_order_note	order_note	Normal	BTREE

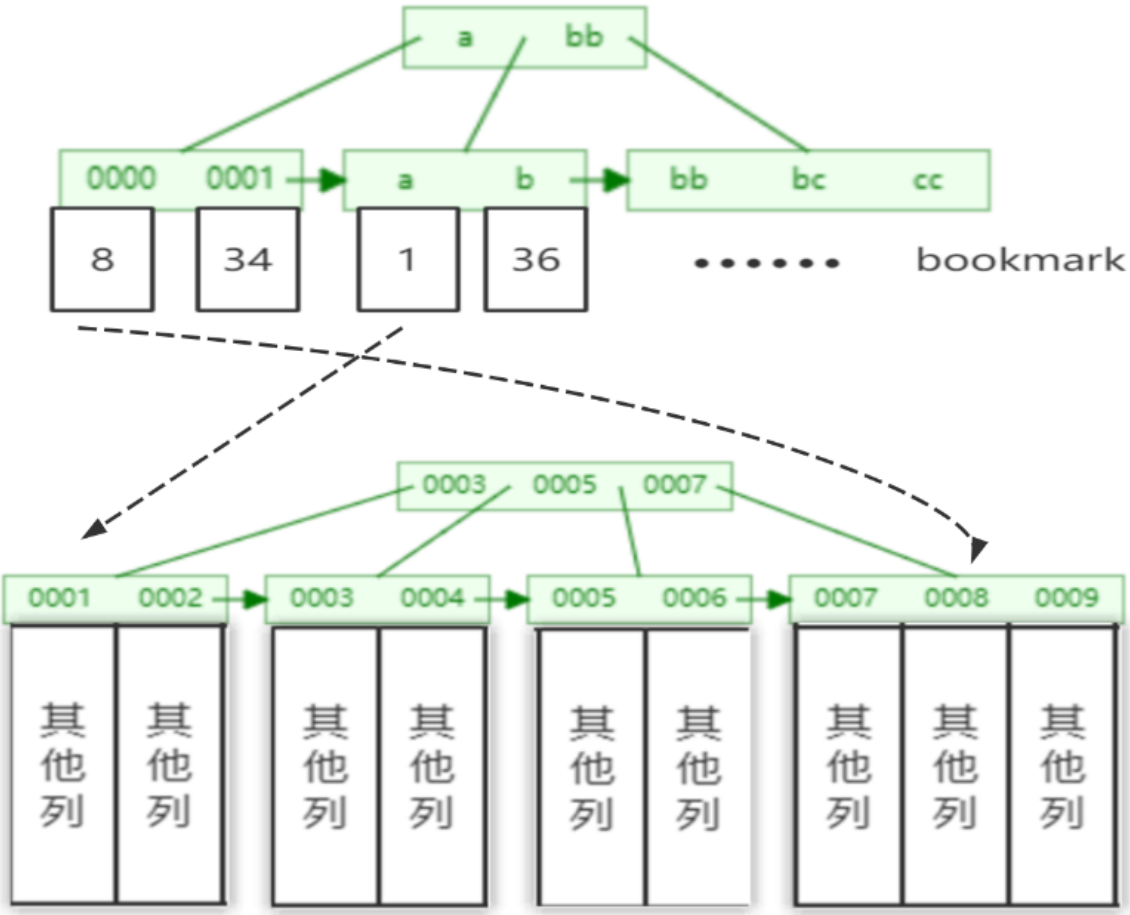
id	order_no	order_note	insert_time	expire_durati	expire_time	order_status
1	1	a	2022-05-09 1	1	2022-05-09 1	1
8	8	0	2022-05-09 1	1	2022-05-09 1	1
34	34	01	2022-05-09 1	1	2022-05-09 1	1
36	36	b	2022-05-09 1	1	2022-05-09 1	1

比如辅助索引index(node)，那么叶子节点中包含的数据就包括了(note和主键)。



1.2.1.3.回表

辅助索引的存在并不影响数据在聚集索引中的组织，因此每张表上可以有多个辅助索引。当通过辅助索引来寻找数据时，InnoDB存储引擎会遍历辅助索引并通过叶级别的指针获得指向主键索引的主键，然后再通过主键索引（聚集索引）来找到一个完整的行记录。这个过程也被称为 **回表** 。也就是根据辅助索引的值查询一条完整的用户记录需要使用到2棵B+树----一次辅助索引，一次聚集索引。



为什么我们还需要一次回表操作呢?直接把完整的用户记录放到辅助索引d的叶子节点不就好了么？如果把完整的用户记录放到叶子节点是可以不用回表，但是太占地方了，相当于每建立一棵B+树都需要把所有的用户记录再都拷贝一遍，这就有点太浪费存储空间了。而且每次对数据的变化要在所有包含数据的索引中全部都修改一次，性能也非常低下。

很明显，回表的记录越少，性能提升就越高，需要回表的记录越多，使用二级索引的性能就越低，甚至让某些查询宁愿使用全表扫描也不使用二级索引。

那什么时候采用全表扫描的方式，什么时候使用采用二级索引 + 回表的方式去执行查询呢？这个就是查询优化器做的工作，查询优化器会事先对表中的记录计算一些统计数据，然后再利用这些统计数据根据查询的条件来计算一下需要回表的记录数，需要回表的记录数越多，就越倾向于使用全表扫描，反之倾向于使用二级索引 + 回表的方式。

1.2.1.4.联合索引/复合索引

前面我们对索引的描述，隐含了一个条件，那就是构建索引的字段只有一个，但实践工作中构建索引的完全可以是多个字段。所以，将表上的多个列组合起来进行索引我们称之为联合索引或者复合索引，比如index(a,b)就是将a,b两个列组合起来构成一个索引。

千万要注意一点，建立联合索引只会建立1棵B+树，多个列分别建立索引会分别以每个列则建立B+树，有几个列就有几个B+树，比如，index(note)、index(b)，就分别对note,b两个列各构建了一个索引。

而如果是index(note,b)在索引构建上，包含了两个意思：

1、先把各个记录按照note列进行排序。

2、在记录的note列相同的情况下，采用b列进行排序

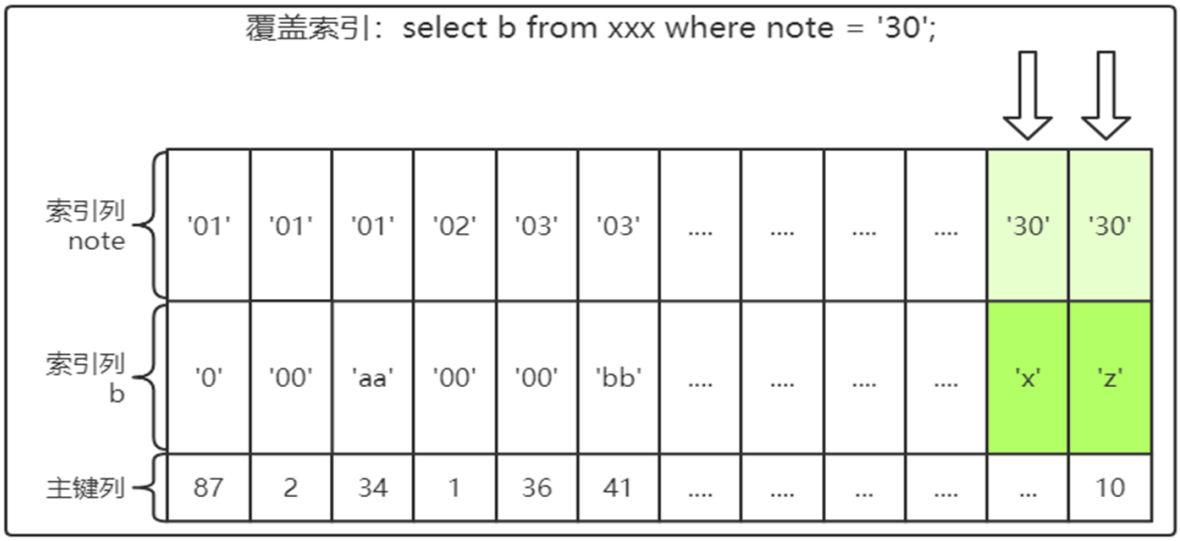
从原理可知，为什么有最佳左前缀法则，就是这个道理

索引列 note	'01'	'01'	'01'	'02'	'03'	'03'	....	....	....	....	'30'	'30'
索引列 b	'0'	'00'	'aa'	'00'	'00'	'bb'	....	....	....	....	'x'	'z'
主键列	87	2	34	1	36	41	....	....	...	....	...	10

1.2.1.5.覆盖索引

既然多个列可以组合起来构建为联合索引，那么辅助索引自然也可以由多个列组成。

InnoDB存储引擎支持覆盖索引(covering index，或称索引覆盖)，即从辅助索引中就可以得到查询的记录，而不需要查询聚集索引中的记录(回表)。使用覆盖索引的一个好处是辅助索引不包含整行记录的所有信息，故其大小要远小于聚集索引，因此可以减少大量的IO操作。所以记住，*覆盖索引并不是索引类型的一种*。



1.2.2.哈希索引

InnoDB存储引擎除了我们前面所说的各种索引，还有一种自适应哈希索引，我们知道B+树的查找次数,取决于B+树的高度,在生产环境中,B+树的高度一般为3、4层,故需要3、4次的IO查询。

所以在InnoDB存储引擎内部自己去监控索引表，如果监控到某个索引经常用，那么就认为是热数据，然后内部自己创建一个hash索引，称之为自适应哈希索引(Adaptive Hash Index,AHI)，创建以后，如果下次又查询到这个索引，那么直接通过hash算法推导出记录的地址，直接一次就能查到数据，比重复去B+tree索引中查询三四次节点的效率高了不少。

InnoDB存储引擎使用的哈希函数采用除法散列方式，其冲突机制采用链表方式。注意，对于自适应哈希索引仅是数据库自身创建并使用的，我们并不能对其进行干预。

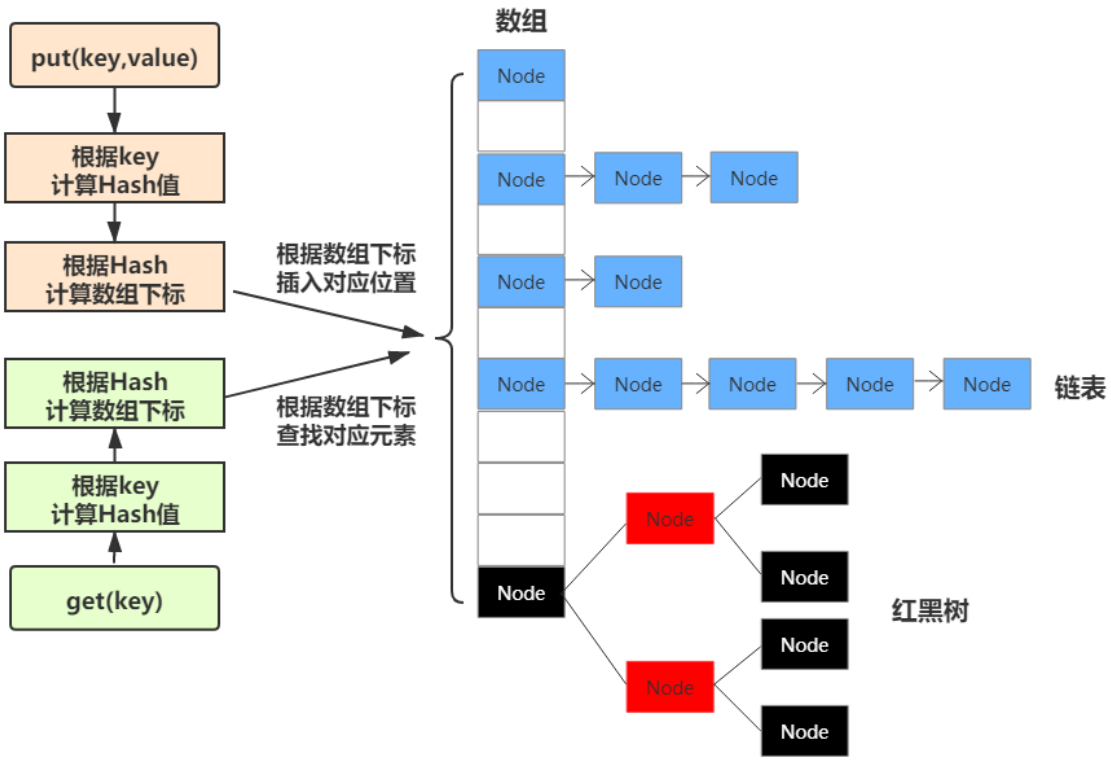
show engine innodb status

```
INSERT BUFFER AND ADAPTIVE HASH INDEX
-------------------------------------
Ibuf: size 1, free list len 0, seg size 2, 0 merges
merged operations:
 insert 0, delete mark 0, delete 0
discarded operations:
 insert 0, delete mark 0, delete 0
Hash table size 2267, node heap has 0 buffer(s)
Hash table size 2267, node heap has 0 buffer(s)
Hash table size 2267, node heap has 0 buffer(s)
Hash table size 2267, node heap has 0 buffer(s)
Hash table size 2267, node heap has 0 buffer(s)
Hash table size 2267, node heap has 0 buffer(s)
Hash table size 2267, node heap has 0 buffer(s)
Hash table size 2267, node heap has 0 buffer(s)
0.00 hash searches/s, 0.00 non-hash searches/s
```

哈希索引只能用来搜索等值的查询,如 SELECT* FROM table WHERE index co=xxx。而对于其他查找类型,如范围查找,是不能使用哈希索引的,

因此这里出现了non-hash searches/s的情况。通过 hash searches: non- hash searches可以大概了解使用哈希索引后的效率。

innodb_adaptive_hash_index来考虑是禁用或启动此特性,默认AHI为开启状态。



1.2.3.全文索引（了解即可）

什么是全文检索（Full-Text Search）？它是将存储于数据库中的整本书或整篇文章中的任意内容信息查找出来的技术。它可以根据需要获得全文中有关章、节、段、句、词等信息，也可以进行各种统计和分析。我们比较熟知的Elasticsearch、Solr等就是全文检索引擎，底层都是基于Apache Lucene的。

举个例子，现在我们要保存唐宋诗词，数据库中我们们会怎么设计？诗词表我们可能的设计如下：

朝代	作者	诗词年代	标题	诗词全文
唐	李白		静夜思	床前明月光，疑是地上霜。 举头望明月，低头思故乡。
宋	李清照		如梦令	常记溪亭日暮，沉醉不知归路，兴尽晚回舟，误入藕花深处。争渡，争渡，惊起一滩鸥鹭。
....	....	...	....	.....

要根据朝代或者作者寻找诗，都很简单，比如“select 诗词全文 from 诗词表 where作者='李白'”，如果数据很多，查询速度很慢，怎么办？我们可以在对应的查询字段上建立索引加速查询。

但是如果我们现在有个需求：要求找到包含“望”字的诗词怎么办？用

“select 诗词全文 from 诗词表 where诗词全文 like‘%望%’”，这个意味着要扫描库中的诗词全文字段，逐条比对，找出所有包含关键词“望”字的记录，。基本上，数据库中一般的SQL优化手段都是用不上的。数量少，大概性能还能接受，如果数据量稍微大点，就完全无法接受了，更何况在互联网这种海量数据的情况下呢？怎么解决这个问题呢，用倒排索引。

倒排索引就是，将文档中包含的关键字全部提取处理，然后再将关键字和文档之间的对应关系保存起来，最后再对关键字本身做索引排序。用户在检索某一个关键字时，先对关键字的索引进行查找，再通过关键字与文档的对应关系找到所在文档。

于是我们可以这么保存

序号	关键字	蜀道难	静夜思	春台望	鹤冲天
1	望	有	有	有	有

如果查哪个诗词中包含上，怎么办，上述的表格可以继续填入新的记录

序号	关键字	蜀道难	静夜思	春台望	鹤冲天
1	望	有	有	有	有
2	上	有			有

从InnoDB 1.2.x版本开始，InnoDB存储引擎开始支持全文检索，对应的MySQL版本是5.6.x系列。不过MySQL从设计之初就是关系型数据库，存储引擎虽然支持全文检索，整体架构上对全文检索支持并不好而且限制很多，比如每张表只能有一个全文检索的索引，不支持没有单词界定符(delimiter）的语言，如中文、日语、韩语等。

所以MySQL中的全文索引功能比较弱鸡，了解即可。

1.2.4.索引在查询中的使用

索引在查询中的作用到底是什么？在我们的查询中发挥着什么样的作用呢？

请记住：

1、 一个索引就是一个B+树，索引让我们的查询可以快速定位和扫描到我们需要的数据记录上，加快查询的速度 。

2、一个select查询语句在执行过程中一般最多能使用一个二级索引，即使在where条件中用了多个二级索引。

1.2.4.高性能的索引创建策略

正确地创建和使用索引是实现高性能查询的基础。前面我们已经了解了索引相关的数据结构，各种类型的索引及其对应的优缺点。现在我们一起来看看如何真正地发挥这些索引的优势。

1.2.4.1.索引列的类型尽量小

我们在定义表结构的时候要显式的指定列的类型，以整数类型为例，有TTNYINT、NEDUMNT、INT、BIGTNT这么几种，它们占用的存储空间依次递增，我们这里所说的类型大小指的就是该类型表示的数据范围的大小。能表示的整数范围当然也是依次递增，如果我们想要对某个整数列建立索引的话，在表示的整数范围允许的情况下，尽量让索引列使用较小的类型，比如我们能使用INT就不要使用BIGINT，能

使用NEDIUMINT就不要使用INT，这是因为数据类型越小，在查询时进行的比较操作越快（CPU层次)数据类型越小，索引占用的存储空间就越少，在一个数据页内就可以放下更多的记录，从而减少磁盘/0带来的性能损耗，也就意味着可以把更多的数据页缓存在内存中，从而加快读写效率。

这个建议对于表的主键来说更加适用，因为不仅是聚簇索引中会存储主键值，其他所有的二级索引的节点处都会存储一份记录的主键值，如果主键适用更小的数据类型，也就意味着节省更多的存储空间和更高效的I/0。

1.2.4.2.索引的选择性

总结来说就是重复度高的列不适合创建索引

创建索引应该选择选择性/离散性高的列。索引的选择性/离散性是指，不重复的索引值（也称为基数，cardinality)和数据表的记录总数（N)的比值，范围从1/N到1之间。索引的选择性越高则查询效率越高，因为选择性高的索引可以让MySQL在查找时过滤掉更多的行。唯一索引的选择性是1，这是最好的索引选择性，性能也是最好的。

很差的索引选择性就是列中的数据重复度很高，比如性别字段，不考虑政治正确的情况下，只有两者可能，男或女。那么我们在查询时，即使使用这个索引，从概率的角度来说，依然可能查出一半的数据出来。

name	sex	age	area
张三	男	18	北京
李四	男	28	上海
王五	女	33	天津
赵六	女	18	北京
孙七	男	12	上海
周八	男	23	深圳
吴九	女	33	广州
郑十	男	23	深圳

哪列做为索引字段最好？当然是姓名字段，因为里面的数据没有任何重复，性别字段是最不适合做索引的，因为数据的重复度非常高。

怎么算索引的选择性/离散性？比如person这个表：

SELECT count(DISTINCT name)/count*() FROM person;*
SELECT count(DISTINCT sex)/count() FROM person;
SELECT count(DISTINCT age)/count*() FROM person;*
SELECT count(DISTINCT area)/count() FROM person;

1.2.4.3.前缀索引

针对blob、text、很长的varchar字段，mysql不支持索引他们的全部长度，需建立前缀索引。

语法：Alter table tableName add key/index (column(X))

缺点：前缀索引是一种能使索引更小、更快的有效办法，但另一方面也有其缺点MySQL无法使用前缀索引做ORDER BY和GROUP BY，也无法使用前缀索引做覆盖扫描。

有时候后缀索引 (suffix index)也有用途（例如，找到某个域名的所有电子邮件地址)。MySQL原生并不支持反向索引，但是可以把字符串反转后存储，并基于此建立前缀索引。可以通过触发器或者应用程序自行处理来维护索引。

案例：

id	order_no	order_note	insert_time	expire_duratio	expire_time	order_status
16	DD00_15S	你好，李焕英。	2021-03-22 18:	15	2021-03-22 18:	0
21	DD00_10S	你好，李焕英。	2021-03-22 18:	10	2021-03-22 18:	0
27	DD00_16S	你好，李焕英。	2021-03-22 18:	16	2021-03-22 18:	0
38	DD00_18S	你好，李焕英。	2021-03-22 18:	18	2021-03-22 18:	0

id	order_no	order_note	insert_time	expire_duratio	expire_time	order_status
2252	DD925_4865	唐探3。DD925_	2021-03-25 22:	48656	2021-03-26 12:	0
2253	DD539_9918	唐探3。DD539	2021-03-25 22:	99186	2021-03-27 02:	0

id	order_no	order_note	insert_time	expire_duratio	expire_time	order_status
5252	DD268_7770	波斯语课。DD2	2021-03-25 23:	77707	2021-03-26 20:	0
5253	DD73_91560	波斯语课。DD7	2021-03-25 23:	91560	2021-03-27 00:	0
5254	DD421_9444	波斯语课。DD4	2021-03-25 23:	94449	2021-03-27 01:	0
5255	DD326_2864	波斯语课。DD3	2021-03-25 23:	28647	2021-03-26 07:	0

首先找到最常见的值的列表：

```
SELECT COUNT(DISTINCT LEFT(order_note,3))/COUNT(*) AS sel3,
COUNT(DISTINCT LEFT(order_note,4))/COUNT(*)AS sel4,
COUNT(DISTINCT LEFT(order_note,5))/COUNT(*) AS sel5,
COUNT(DISTINCT LEFT(order_note, 6))/COUNT(*) As sel6,
COUNT(DISTINCT LEFT(order_note, 7))/COUNT(*) As sel7,
COUNT(DISTINCT LEFT(order_note, 8))/COUNT(*) As sel8,
COUNT(DISTINCT LEFT(order_note, 9))/COUNT(*) As sel9,
COUNT(DISTINCT LEFT(order_note, 10))/COUNT(*) As sel10,
COUNT(DISTINCT LEFT(order_note, 11))/COUNT(*) As sel11,
COUNT(DISTINCT LEFT(order_note, 12))/COUNT(*) As sel12,
COUNT(DISTINCT LEFT(order_note, 13))/COUNT(*) As sel13,
COUNT(DISTINCT LEFT(order_note, 14))/COUNT(*) As sel14,
COUNT(DISTINCT LEFT(order_note, 15))/COUNT(*) As sel15,
COUNT(DISTINCT order_note)/COUNT(*) As total
FROM order_exp;
```

信息 结果1 概况 状态

sel3	sel4	sel5	sel6	sel7	sel8	sel9	sel10	sel11	sel12	sel13	sel14	sel15	total
0.0008	0.0008	0.0008	0.0015	0.0107	0.0844	0.1628	0.3455	0.4723	0.6834	0.8564	0.9197	0.9592	0.9676

可以看见，从第10个开始选择性的增加值很高，随着前缀字符的越来越多，选择度也在不断上升，但是增长到第15时，已经和第14没太大差别了，选择性提升的幅度已经很小了，都非常接近整个列的选择性了。

那么针对这个字段做前缀索引的话，从第13到第15都是不错的选择

在上面的示例中，已经找到了合适的前缀长度，如何创建前缀索引:

ALTER TABLE order_exp ADD KEY (order_note(14));

建立前缀索引后查询语句并不需要更改:

select * from order_exp where order_note = 'xxxx' ;

前缀索引是一种能使索引更小、更快的有效办法，但另一方面也有其缺点MySQL无法使用前缀索引做ORDER BY和GROUP BY，也无法使用前缀索引做覆盖扫描。

有时候后缀索引 (suffix index)也有用途（例如，找到某个域名的所有电子邮件地址)。MySQL原生并不支持反向索引，但是可以把字符串反转后存储，并基于此建立前缀索引。可以通过触发器或者应用程序自行处理来维护索引。

1.2.4.4.只为用于搜索、排序或分组的列创建索引

也就是说，只为出现在WHERE 子句中的列、连接子句中的连接列创建索引，而出现在查询列表中的列一般就没必要建立索引了，除非是需要使用覆盖索引；又或者为出现在ORDER BY或GROUP BY子句中的列创建索引，这句话什么意思呢？比如:

搜索

select order_note from .... and ....

只为 条件中的列建立索引即可

排序

SELECT * FROM order_exp ORDER BY insert_time, order_status,expire_time;

```
1 SELECT * FROM order_exp ORDER BY insert_time, order_status,expire_time;
```

结果1 概况 状态

d	order_no	order_note	insert_time	expire_duratio	expire_time	order_status
16	DD00_15S	你好，李焕英。	2021-03-22 18:	15	2021-03-22 18:	0

查询的结果集需要先按照insert_time值排序，如果记录的insert_time值相同，则需要按照order_status来排序，如果order_status的值相同，则需要按照expire_time排序。回顾一下联合索引的存储结构，u_idx_day_status索引本身就是按照上述规则排好序的，所以直接从索引中提取数据，然后进行回表操作取出该索引中不包含的列就好了。

栏位 索引 外键 触发器 选项 注释 SQL 预览

名	栏位	索引类型	索引方法
u_idx_day_status	insert_time, order_status, expire_time	Unique	BTREE
idx_order_no	order_no	Normal	BTREE

1.2.4.5.多列索引

很多人对多列索引的理解都不够。一个常见的错误就是，为每个列创建独立的索引，或者按照错误的顺序创建多列索引。

我们遇到的最容易引起困惑的问题就是索引列的顺序。正确的顺序依赖于使用该索引的查询，并且同时需要考虑如何更好地满足排序和分组的需要。反复强调过，在一个多列B-Tree索引中，索引列的顺序意味着索引首先按照最左列进行排序，其次是第二列，等等。所以，索引可以按照升序或者降序进行扫描，以满足精确符合列顺序的ORDER BY、GROUP BY和DISTINCT等子句的查询需求。

所以多列索引的列顺序至关重要。对于如何选择索引的列顺序有一个经验法则：将选择性最高的列放到索引最前列。当不需要考虑排序和分组时，将选择性最高的列放在前面通常是很好的。这时候索引的作用只是用于优化WHERE条件的查找。在这种情况下，这样设计的索引确实能够最快地过滤出需要的行，对于在WHERE子句中只使用了索引部分前缀列的查询来说选择性也更高。

然而，性能不只是依赖于索引列的选择性，也和查询条件的有关。可能需要根据那些运行频率最高的查询来调整索引列的顺序，比如排序和分组，让这种情况下索引的选择性最高。

同时，在优化性能的时候，可能需要使用相同的列但顺序不同的索引来满足不同类型的查询需求。

1.2.4.6.三星索引

三星索引概念

对于一个查询而言，一个三星索引，可能是其最好的索引。

满足的条件如下：

- 索引将相关的记录放到一起则获得一星　（比重27%）
- 如果索引中的数据顺序和查找中的排列顺序一致则获得二星（排序星）　（比重23%）
- 如果索引中的列包含了查询中需要的全部列则获得三星（宽索引星）　（比重50%）

这三颗星，哪颗最重要？第三颗星。因为将一个列排除在索引之外可能会导致很多磁盘随机读（回表操作）。第一和第二颗星重要性差不多，可以理解为第三颗星比重是50%，第一颗星为27%，第二颗星为23%，所以在大部分的情况下，会先考虑第一颗星，但会根据业务情况调整这两颗星的优先度。

一星：

一星的意思就是：如果一个查询相关的索引行是相邻的或者至少相距足够靠近的话，必须扫描的索引片宽度就会缩至最短，也就是说，让索引片尽量变窄，也就是我们所说的索引的扫描范围越小越好。

二星（排序星）：

在满足一星的情况下，当查询需要排序，group by、 order by，如果查询所需的顺序与索引是一致的（索引本身是有序的），是不是就可以不用再另外排序了，一般来说排序可是影响性能的关键因素。

三星（宽索引星）：

在满足了二星的情况下，如果索引中所包含了这个查询所需的所有列（包括 where 子句和 select 子句中所需的列，也就是覆盖索引），这样一来，查询就不再需要回表了，减少了查询的步骤和IO请求次数，性能几乎可以提升一倍。

1.2.4.6.设计三星索引实战

现在有表，SQL如下

```
CREATE TABLE customer (
    cno INT,
    lname VARCHAR (10),
    fname VARCHAR (10),
    sex INT,
    weight INT,
    city VARCHAR (10)
);

CREATE INDEX idx_cust ON customer (city, lname, fname, cno);
```

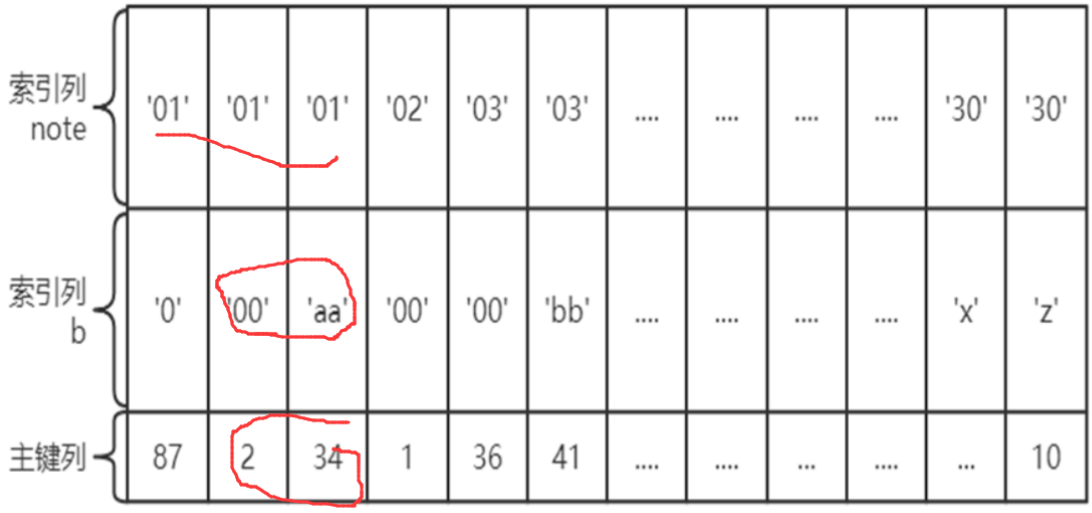
对于下面的SQL而言，这是个三星索引

```
select cno,fname from customer where lname='xx' and city ='yy' order by fname;
```

来评估下：

第一颗星：所有等值谓词的列，是组合索引的开头的列，可以把索引片缩得很窄，符合。

根据之前讲过的联合索引，我们是知道条件已经把搜索范围搜到很窄了



第二颗星：order by的fname字段在组合索引中且是索引自动排序好的，符合。

第三颗星：select中的cno字段、fname字段在组合索引中存在，符合。

```
CREATE INDEX idx_cust ON customer (city, lname, fname, cno);
```

对于下面的SQL而言，这是个三星索引

```
select cno,fname from customer where lname='xx' and city ='yy' order by fname;
```

现在有表，SQL如下：

```
CREATE TABLE `test` (
    `id` INT (11) NOT NULL AUTO_INCREMENT,
    `user_name` VARCHAR (100) DEFAULT NULL,
    `sex` INT (11) DEFAULT NULL,
    `age` INT (11) DEFAULT NULL,
    `c_date` datetime DEFAULT NULL,
    PRIMARY KEY (`id`),

) ENGINE = INNODB AUTO_INCREMENT = 12 DEFAULT CHARSET = utf8;
```

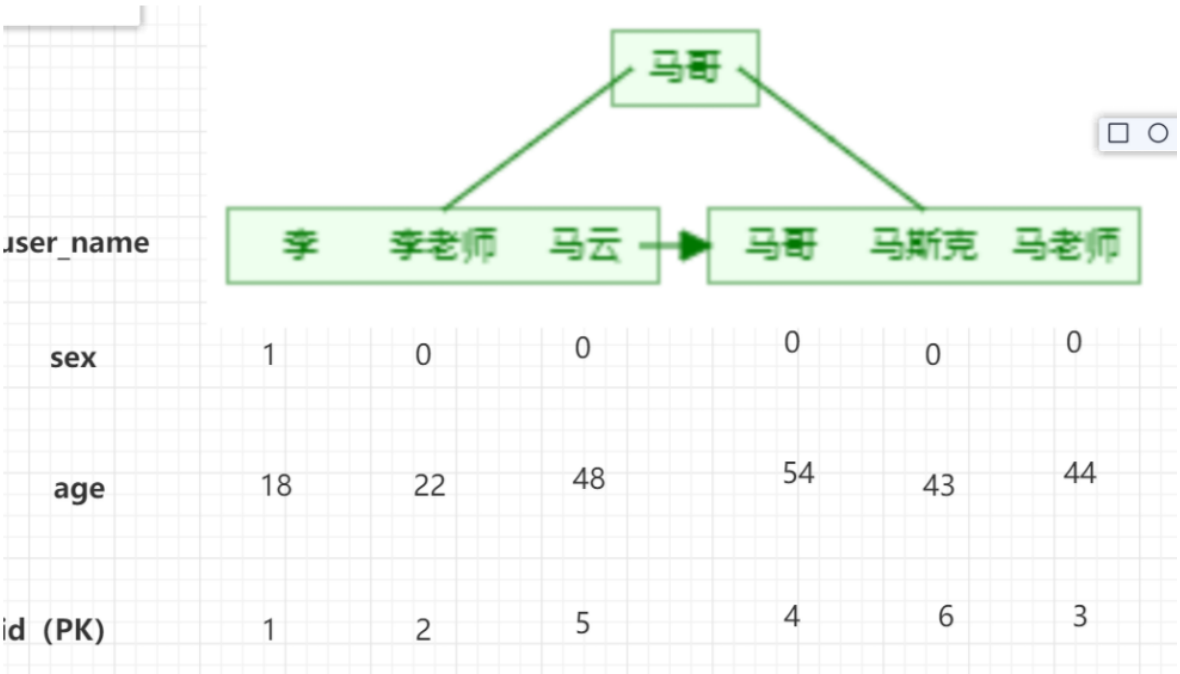
SQL语句如下：

```
select user_name,sex,age from test where user_name like 'test%'  and sex =1
ORDER BY age
```

如果我们建立索引(user_name,sex,age)：

id	user_name	sex	age	c_date
1	李	1	18	2022-05-10 21:57:50
2	李老师	0	22	2022-05-10 21:57:50
3	马老师	0	44	2022-05-10 21:57:50
4	马哥	0	54	2022-05-10 21:57:50
5	马云	0	48	2022-05-10 21:57:50
6	马斯克	0	43	2022-05-10 21:57:50

```sql
SELECT
  user_name,
  sex,
  age
FROM
  test
WHERE
  user_name LIKE '马%'
AND sex = 0
ORDER BY
  age
```



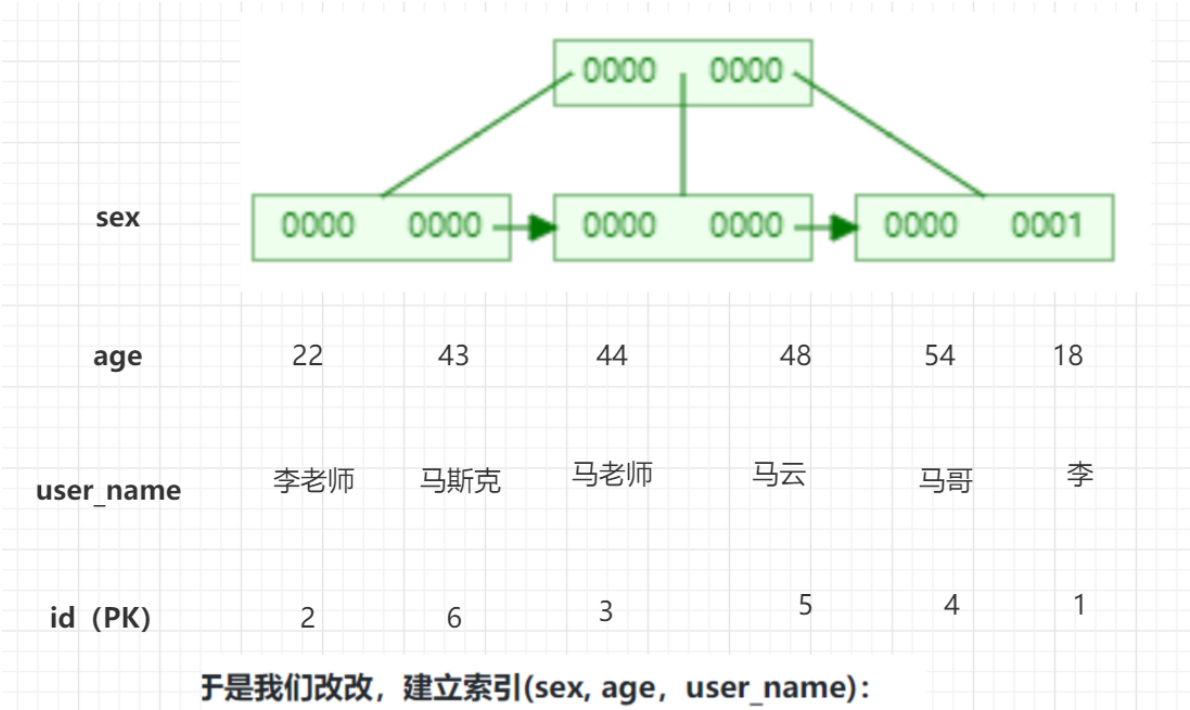
第三颗星，满足

第一颗星，满足

第二颗星，不满足，user_name 采用了范围匹配，sex 是过滤列，此时age 列无法保证有序的。

上述我们看到，此时索引(user_name,sex,age)并不能满足三星索引中的第二颗星（排序）。

于是我们改改，建立索引(sex, age，user_name)：



第一颗星，不满足，只可以匹配到sex，sex选择性很差，意味着是一个宽索引片(同时因为age也会导致排序选择的碎片问题)

第二颗星，满足，等值sex 的情况下，age是有序的，

第三颗星，满足，select查询的列都在索引列中，

对于索引(sex,age，user_name)我们可以看到，此时无法满足第一颗星，窄索引片的需求。

以上2个索引，都是无法同时满足三星索引设计中的三个需求的，我们只能尽力满足2个。而在多数情况下，能够满足2颗星，已经能缩小很大的查询范围了，具体最终要保留那一颗星（排序星 or 窄索引片星），这个就需要看查询者自己的着重点了，无法给出标准答案。